# DSA Course Lecture: Java Basics

**Easy notes from your Hindi YouTube lecture (~1 hour 26 minutes)**
This particular lecture teaches Java foundations needed before solving DSA problems. It is not yet a full lesson on DSA algorithms.

## 1. Operators (about 0:00-11:30)

An **operator** is a symbol that performs an action on values. Example: **5 + 3** gives **8**. Here + is the operator; 5 and 3 are operands.

**Arithmetic:** +, -, *, /, % (remainder). **Comparison:** ==, !=, >, <, >=, <=. **Logical:** && (AND), || (OR), ! (NOT). **Assignment:** =, +=, -=. **Unary:** ++, --, !. The lecture also introduces bitwise and shift operators.

```
int a = 10, b = 3;
System.out.println(a + b); // 13
System.out.println(a % b); // 1
System.out.println(a > b); // true
```

**Ternary operator:** a short if/else expression: **condition ? valueIfTrue : valueIfFalse**.

```
int age = 20;
String result = age >= 18 ? "Adult" : "Minor";
```

## 2. Conditions: if, else, else-if (about 11:30-26:00)

A **condition** helps a program make a decision. If the test is true, one block runs; otherwise another block can run.

```
int marks = 72;
if (marks >= 80) {
    System.out.println("Excellent");
} else if (marks >= 40) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

**Remember:** Use **if** for one check, **if-else** for two paths, and **else-if ladder** when there are multiple conditions. A nested if is an if inside another if.

## 3. switch statement (about 26:00-35:00)

A **switch** is convenient when one value can match several fixed cases, such as day numbers. **break** stops execution from falling into the next case in a traditional switch.

```
int day = 2;
switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    default: System.out.println("Unknown day");
}
```

## 4. Loops: repeat work (about 35:00-49:00)

A **loop** repeats code instead of writing the same statement many times. **for**: useful when the number of repeats is known. **while**: checks before each run. **do-while**: runs at least once because it checks after the body.

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i); // prints 1 to 5
```

```
}

int n = 1;
while (n <= 3) {
    System.out.println(n);
    n++;
}
```

A **for-each** loop visits each item of an array or collection.

```
int[] nums = {2, 4, 6};
for (int x : nums) {
    System.out.println(x);
}
```

## 5. break and continue (about 49:00-54:00)

**break** exits the loop (or switch). **continue** skips the rest of the current loop iteration and moves to the next one.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue; // skip 3
    if (i == 5) break;    // stop at 5
    System.out.println(i); // 1, 2, 4
}
```

## 6. Class and object (about 54:00-1:04:00)

A **class** is a blueprint. An **object** is a real instance made from that blueprint. An object can store **data/state** (fields) and perform **actions/behavior** (methods).

```
class Student {
    String name;
    void study() { System.out.println(name + " studies"); }
}

Student s1 = new Student();
s1.name = "Aman";
s1.study();
```

## 7. Constructor and this keyword (about 1:04:00-1:16:00)

A **constructor** initializes an object when you create it using **new**. It has the same name as the class and no return type. **this** refers to the current object, helping distinguish its fields from parameters.

```
class Student {
    String name;
    Student(String name) {
        this.name = name;
    }
}
Student s = new Student("Aman");
```

**Constructor overloading:** You can have more than one constructor with different parameter lists.

## 8. Method overloading (about 1:15:00-1:19:00)

**Overloading** means methods have the same name but different parameters (different count or types). Java chooses the matching method based on arguments.

```
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
```

## 9. Inheritance, overriding and super (about 1:19:00-1:25:00)

**Inheritance** lets a child class reuse accessible features of a parent class using **extends**. **Overriding** means a child class provides its own version of an inherited method. **super** refers to the parent-class implementation or members.

```
class Animal {
    void eat() { System.out.println("Animal eats"); }
}
class Dog extends Animal {
    @Override
    void eat() {
        super.eat();
        System.out.println("Dog eats");
    }
}
```

**Overloading vs overriding:** Overloading = same name, different parameters. Overriding = child changes an inherited method with a matching signature.

## 10. final keyword (about 1:25:00-end)

**final variable:** cannot be reassigned after initialization. **final method:** cannot be overridden. **final class:** cannot be extended.

```
final int MAX = 100;
// MAX = 200; // compile error
final class Utility { }
```

## Quick revision: remember in one minute

• Operator = do a calculation or comparison.

• if/switch = make a decision.

• Loop = repeat a task.

• break = stop; continue = skip this turn.

• Class = blueprint; object = instance.

• Constructor = set up a new object; this = current object.

• Overloading = same name, different inputs.

• Inheritance = child reuses parent; overriding = child changes behavior.

• super = parent; final = restrict reassignment/overriding/inheritance.

## Practice questions (try in Java)

1. Find whether a number is even or odd using %.

2. Print the largest of three numbers using if-else.

3. Print day names using switch.

4. Print 1 to 100 using a for loop.

5. Calculate the sum of numbers 1 to 10 using a loop.

6. Print an array using for-each.

7. Create a Student class with name and age.

8. Add a constructor using this.name.

9. Overload an add() method.

10. Create Animal and Dog classes; override a method.

**Source note:** These notes simplify the concepts discussed in your supplied Hindi transcript, with newly written Java examples for learning. The exercise questions are additional practice, not quoted from the lecture.